

The News Summarization Service was developed to turn long English news articles into shorter summaries that are easier to read. A user can paste an article into the service and receive a summary of its main points. The project focuses on using an existing language model, building a working API around it, and checking how well its summaries match those written by people.

The model used is DistilBART CNN from Hugging Face, which is already trained for news summarization. It runs locally using Python and PyTorch, with the dependencies installed in a Conda environment. Before an article reaches the model, the text is cleaned to remove unnecessary spaces and invisible characters while keeping its punctuation and capitalization. Articles longer than 1,024 model tokens are shortened to fit the input limit, and the response tells the user when this happens. No additional training or fine-tuning was carried out.

FastAPI provides the interface for submitting articles and receiving summaries. The response includes the generated text, token counts, processing time, and whether the article was truncated. The API also has an interactive documentation page where users can try an example or enter their own article. The same source code is used by the API, the evaluation program, and the demonstration notebook.

The service was evaluated on 500 randomly selected articles from the official CNN/DailyMail 3.0.0 test split. A fixed random seed was used so that the same sample can be selected again. Each generated summary was compared with the article's human-written highlights using ROUGE. On a scale of 0 to 100, the average F1 scores were 44.66 for ROUGE-1, 21.03 for ROUGE-2, and 30.88 for ROUGE-L. Average processing time was about 0.86 seconds per article on an RTX 4060 Laptop GPU after the model had loaded.

The completed project includes the source code, an executed notebook, sample articles and summaries, evaluation results, and setup instructions. All 25 tests passed, and the eight notebook code cells ran without errors. The service was checked on both GPU and CPU, and requests to the running API were tested. Dataset and model versions, package versions, and evaluation settings were recorded to make the work reproducible. Development was saved through local Git commits.

There are still limits to the summaries. In the evaluation sample, 142 articles, or 28.4%, exceeded the input limit, so information near the end could be lost. Reviewing sample outputs also revealed omitted details, a wrong team attribution, and an unfinished sentence. ROUGE measures overlap with a reference summary rather than factual accuracy, so generated claims still need to be checked against the original article. The project demonstrates a working local summarization service; URL scraping, multilingual support, and production deployment were outside its scope.